exercise3

March 29, 2026

1 Exercise 3

2 3.1

[3]:
```
from tensorflow.keras.datasets import mnist
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt

(train_images, train_labels), _ = mnist.load_data()
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype('float32') / 255

alpha = 1.
model = keras.Sequential([
  layers.Dense(512, activation='relu'),
  layers.Dense(10, activation='softmax')
])
model.compile(optimizer=keras.optimizers.RMSprop(alpha),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
history=model.fit(train_images, train_labels,
          epochs=10,
          batch_size=128,
          validation_split=0.2)


accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

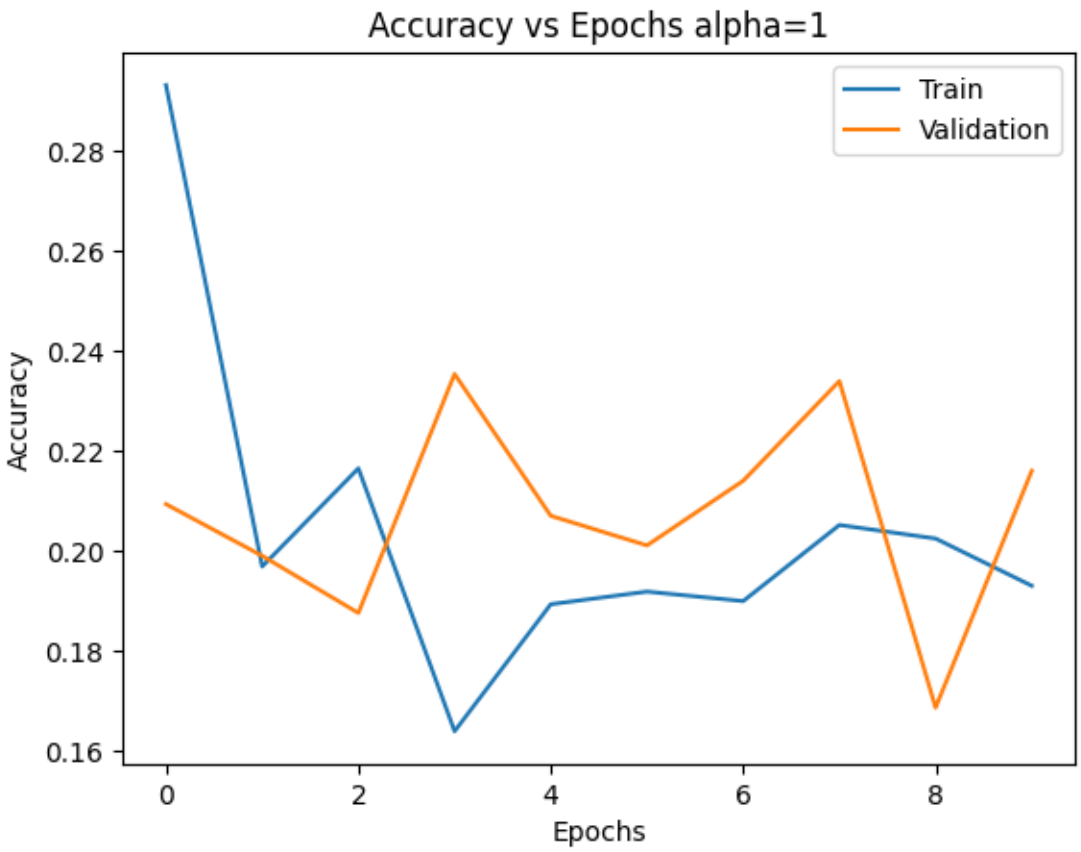
plt.figure()
plt.plot(range(len(accuracy)), accuracy)
plt.plot(range(len(val_accuracy)), val_accuracy)
plt.title("Accuracy vs Epochs alpha=1")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
```

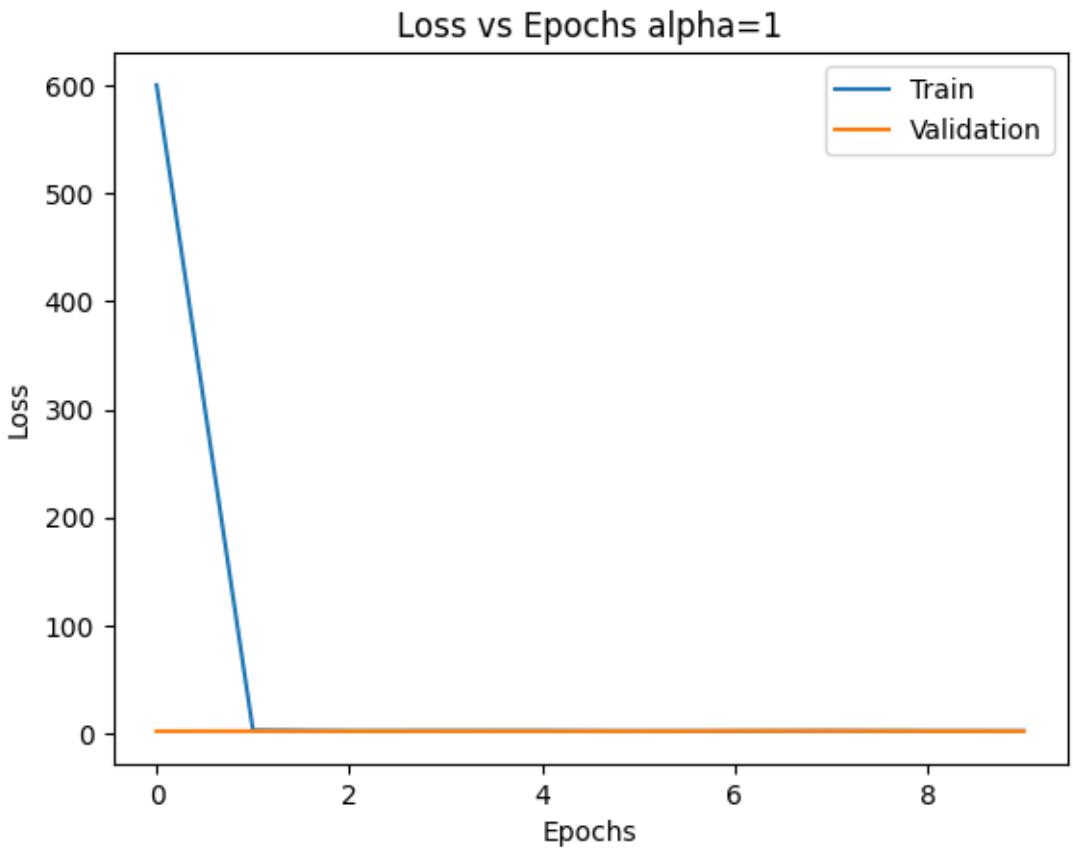
```
plt.legend(["Train", "Validation"])
plt.show()

loss = history.history['loss']
val_loss = history.history['val_loss']

plt.figure()
plt.plot(range(len(loss)), loss)
plt.plot(range(len(val_loss)), val_loss)
plt.title("Loss vs Epochs alpha=1")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

```
Epoch 1/10
375/375             2s 4ms/step -
accuracy: 0.2931 - loss: 600.0192 - val_accuracy: 0.2093 - val_loss: 2.1284
Epoch 2/10
375/375             1s 3ms/step -
accuracy: 0.1969 - loss: 3.0967 - val_accuracy: 0.1990 - val_loss: 2.2350
Epoch 3/10
375/375             1s 3ms/step -
accuracy: 0.2165 - loss: 2.5080 - val_accuracy: 0.1876 - val_loss: 2.2164
Epoch 4/10
375/375             1s 3ms/step -
accuracy: 0.1640 - loss: 2.6019 - val_accuracy: 0.2353 - val_loss: 2.1779
Epoch 5/10
375/375             1s 3ms/step -
accuracy: 0.1893 - loss: 2.5877 - val_accuracy: 0.2070 - val_loss: 2.2076
Epoch 6/10
375/375             1s 3ms/step -
accuracy: 0.1919 - loss: 2.3528 - val_accuracy: 0.2011 - val_loss: 2.1445
Epoch 7/10
375/375             1s 3ms/step -
accuracy: 0.1900 - loss: 2.3672 - val_accuracy: 0.2140 - val_loss: 2.4447
Epoch 8/10
375/375             1s 3ms/step -
accuracy: 0.2052 - loss: 2.5514 - val_accuracy: 0.2339 - val_loss: 2.4633
Epoch 9/10
375/375             1s 3ms/step -
accuracy: 0.2025 - loss: 2.4150 - val_accuracy: 0.1687 - val_loss: 2.2960
Epoch 10/10
375/375             1s 3ms/step -
accuracy: 0.1930 - loss: 2.3976 - val_accuracy: 0.2160 - val_loss: 2.1202
```


Accuracy vs Epochs alpha=1
Train
Validation
Accuracy
Epochs
0.28
0.26
0.24
0.22
0.20
0.18
0.16
0
2
4
6
8



[4]:
```
alpha = 0.5
model = keras.Sequential([
  layers.Dense(512, activation='relu'),
  layers.Dense(10, activation='softmax')
])
model.compile(optimizer=keras.optimizers.RMSprop(alpha),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
history=model.fit(train_images, train_labels,
          epochs=10,
          batch_size=128,
          validation_split=0.2)

accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

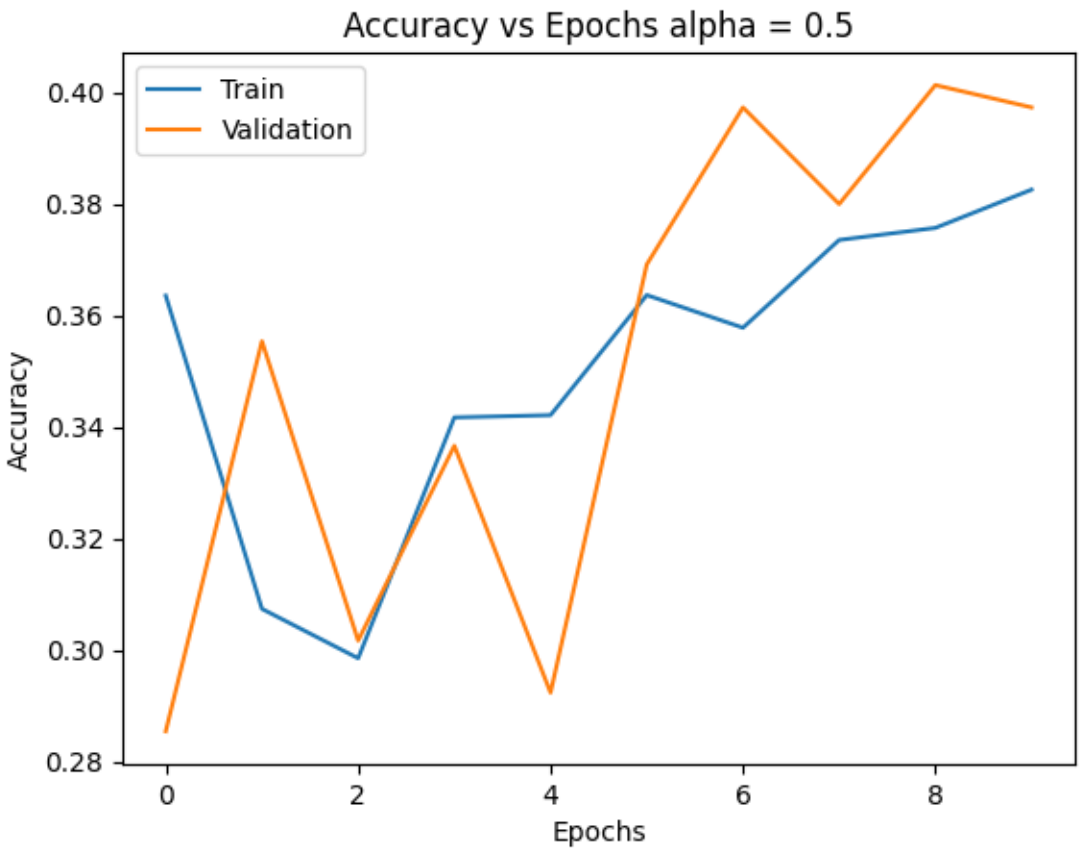
plt.figure()
plt.plot(range(len(accuracy)), accuracy)
plt.plot(range(len(val_accuracy)), val_accuracy)
```

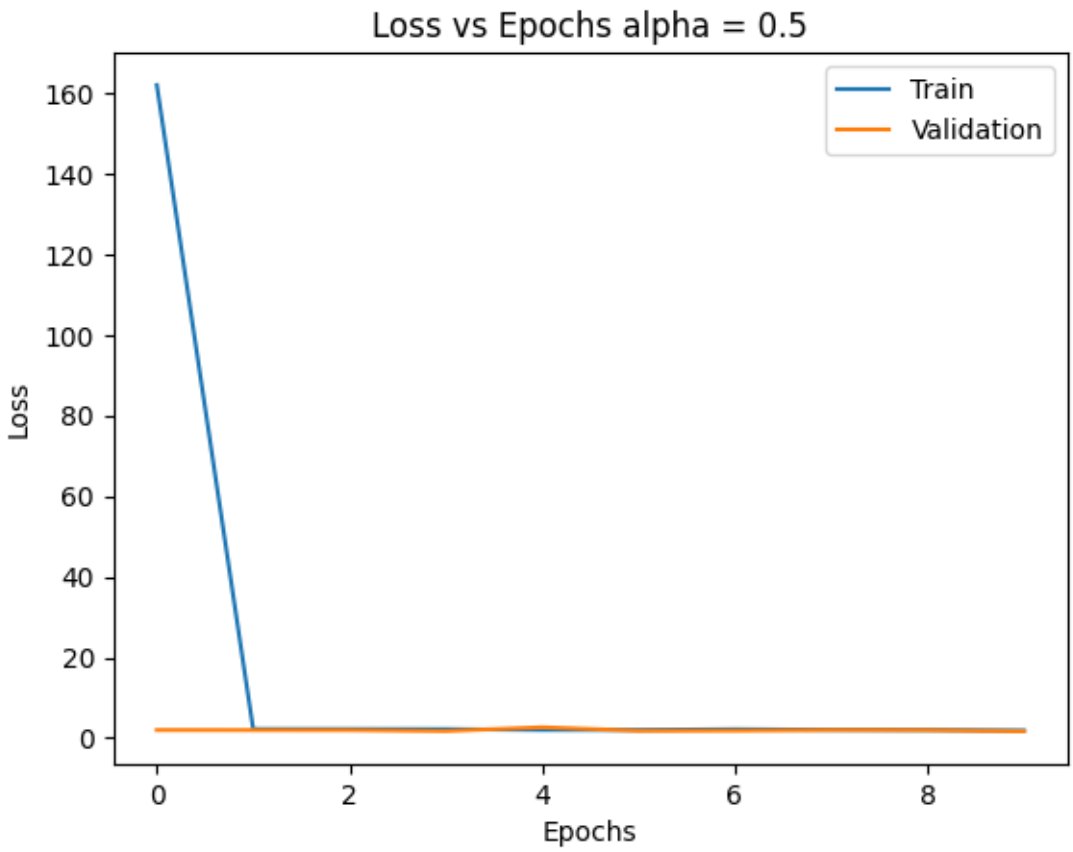
```
plt.title("Accuracy vs Epochs alpha = 0.5")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

loss = history.history['loss']
val_loss = history.history['val_loss']

plt.figure()
plt.plot(range(len(loss)), loss)
plt.plot(range(len(val_loss)), val_loss)
plt.title("Loss vs Epochs alpha = 0.5")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

```
Epoch 1/10
375/375             2s 4ms/step -
accuracy: 0.3635 - loss: 161.9478 - val_accuracy: 0.2854 - val_loss: 1.9878
Epoch 2/10
375/375             1s 3ms/step -
accuracy: 0.3074 - loss: 2.2241 - val_accuracy: 0.3553 - val_loss: 1.9755
Epoch 3/10
375/375             1s 3ms/step -
accuracy: 0.2985 - loss: 2.2203 - val_accuracy: 0.3017 - val_loss: 1.9603
Epoch 4/10
375/375             1s 3ms/step -
accuracy: 0.3417 - loss: 2.2081 - val_accuracy: 0.3366 - val_loss: 1.7441
Epoch 5/10
375/375             1s 3ms/step -
accuracy: 0.3421 - loss: 2.0079 - val_accuracy: 0.2923 - val_loss: 2.6522
Epoch 6/10
375/375             1s 3ms/step -
accuracy: 0.3636 - loss: 2.0120 - val_accuracy: 0.3691 - val_loss: 1.8244
Epoch 7/10
375/375             1s 3ms/step -
accuracy: 0.3577 - loss: 2.1817 - val_accuracy: 0.3972 - val_loss: 1.8347
Epoch 8/10
375/375             1s 3ms/step -
accuracy: 0.3735 - loss: 2.0514 - val_accuracy: 0.3799 - val_loss: 1.9837
Epoch 9/10
375/375             1s 3ms/step -
accuracy: 0.3756 - loss: 1.9918 - val_accuracy: 0.4013 - val_loss: 1.9524
Epoch 10/10
375/375             1s 3ms/step -
accuracy: 0.3825 - loss: 1.8605 - val_accuracy: 0.3972 - val_loss: 1.6835
```


Accuracy vs Epochs alpha = 0.5
Train
Validation
Accuracy
Epochs
0.40
0.38
0.36
0.34
0.32
0.30
0.28
0
2
4
6
8



[5]:
```
alpha = 0.1
model = keras.Sequential([
  layers.Dense(512, activation='relu'),
  layers.Dense(10, activation='softmax')
])
model.compile(optimizer=keras.optimizers.RMSprop(alpha),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
history=model.fit(train_images, train_labels,
          epochs=10,
          batch_size=128,
          validation_split=0.2)

accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

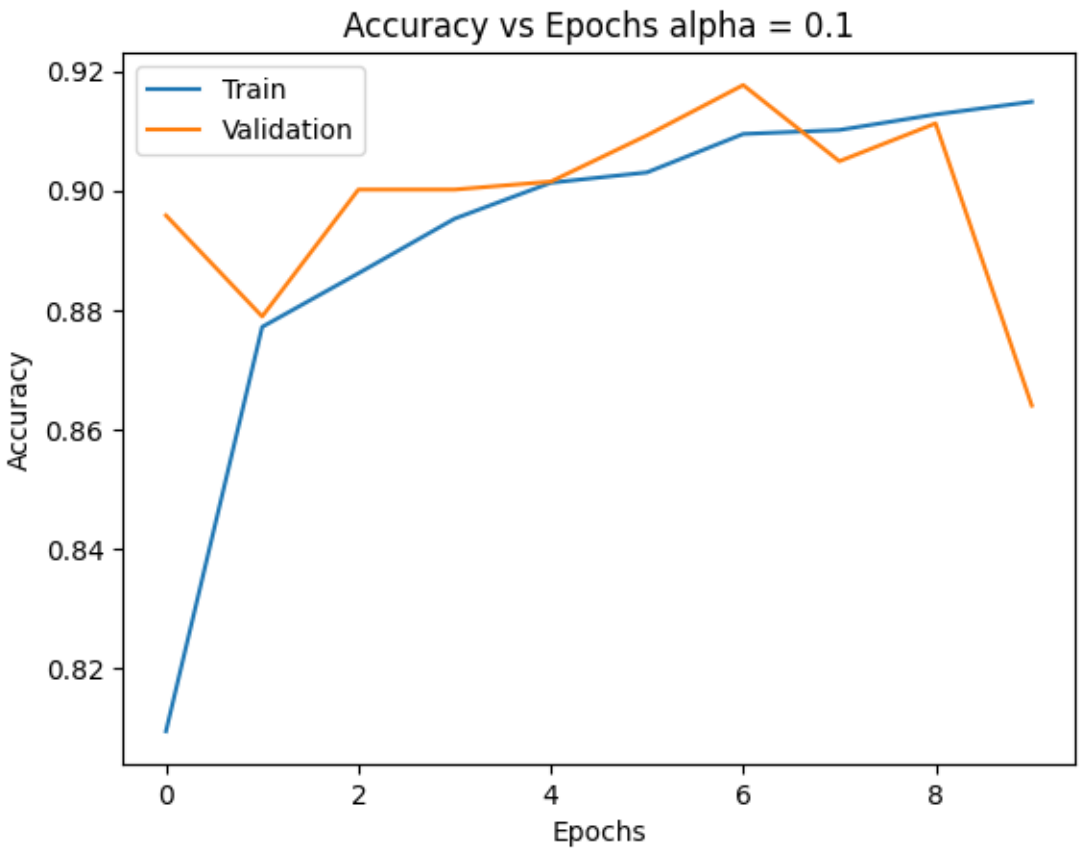
plt.figure()
plt.plot(range(len(accuracy)), accuracy)
plt.plot(range(len(val_accuracy)), val_accuracy)
```

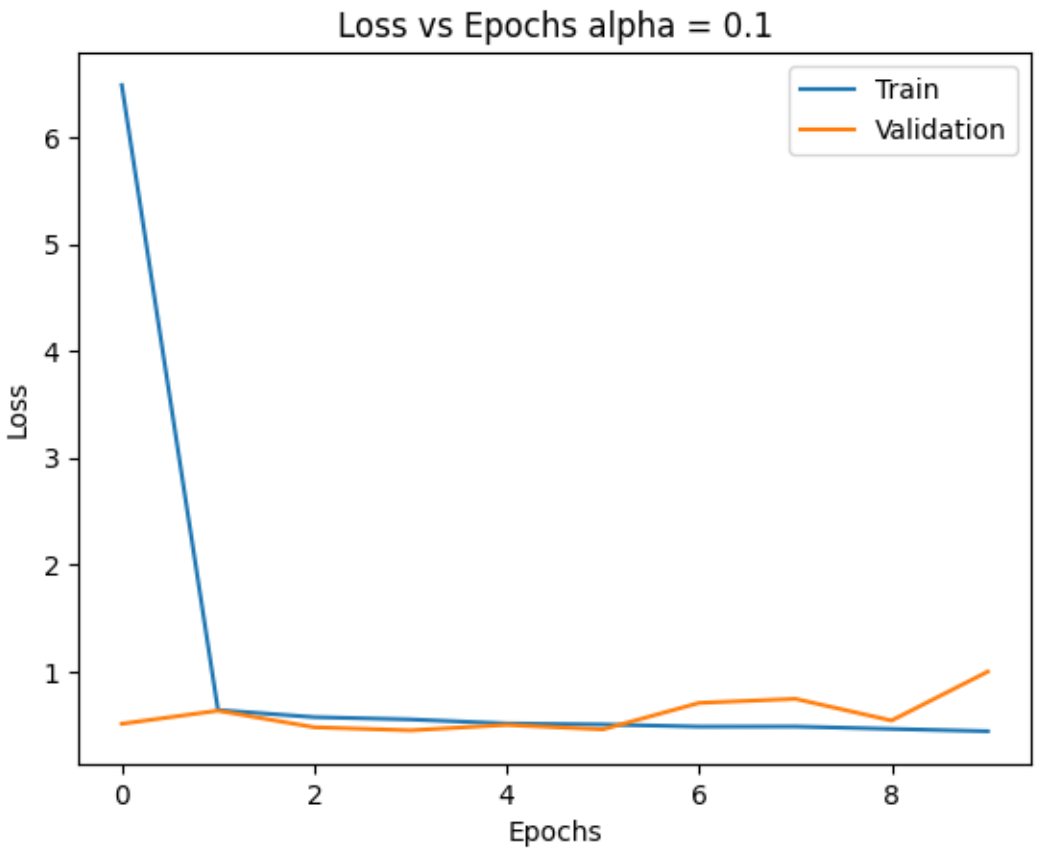
```
plt.title("Accuracy vs Epochs alpha = 0.1")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

loss = history.history['loss']
val_loss = history.history['val_loss']

plt.figure()
plt.plot(range(len(loss)), loss)
plt.plot(range(len(val_loss)), val_loss)
plt.title("Loss vs Epochs alpha = 0.1")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

```
Epoch 1/10
375/375             2s 4ms/step -
accuracy: 0.8095 - loss: 6.4904 - val_accuracy: 0.8958 - val_loss: 0.5103
Epoch 2/10
375/375             1s 3ms/step -
accuracy: 0.8772 - loss: 0.6386 - val_accuracy: 0.8789 - val_loss: 0.6330
Epoch 3/10
375/375             1s 3ms/step -
accuracy: 0.8861 - loss: 0.5726 - val_accuracy: 0.9002 - val_loss: 0.4778
Epoch 4/10
375/375             1s 3ms/step -
accuracy: 0.8953 - loss: 0.5511 - val_accuracy: 0.9002 - val_loss: 0.4491
Epoch 5/10
375/375             1s 3ms/step -
accuracy: 0.9013 - loss: 0.5128 - val_accuracy: 0.9015 - val_loss: 0.5001
Epoch 6/10
375/375             1s 3ms/step -
accuracy: 0.9030 - loss: 0.5038 - val_accuracy: 0.9093 - val_loss: 0.4582
Epoch 7/10
375/375             1s 3ms/step -
accuracy: 0.9095 - loss: 0.4840 - val_accuracy: 0.9177 - val_loss: 0.7058
Epoch 8/10
375/375             1s 3ms/step -
accuracy: 0.9101 - loss: 0.4847 - val_accuracy: 0.9049 - val_loss: 0.7437
Epoch 9/10
375/375             1s 3ms/step -
accuracy: 0.9128 - loss: 0.4623 - val_accuracy: 0.9112 - val_loss: 0.5409
Epoch 10/10
375/375             1s 3ms/step -
accuracy: 0.9149 - loss: 0.4407 - val_accuracy: 0.8640 - val_loss: 0.9988
```


Accuracy vs Epochs alpha = 0.1
Train
Validation
0.92
0.90
0.88
0.86
0.84
0.82
Accuracy
0
2
4
6
8
Epochs



[6]:
```
alpha = 0.01
model = keras.Sequential([
  layers.Dense(512, activation='relu'),
  layers.Dense(10, activation='softmax')
])
model.compile(optimizer=keras.optimizers.RMSprop(alpha),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
history=model.fit(train_images, train_labels,
          epochs=10,
          batch_size=128,
          validation_split=0.2)

accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

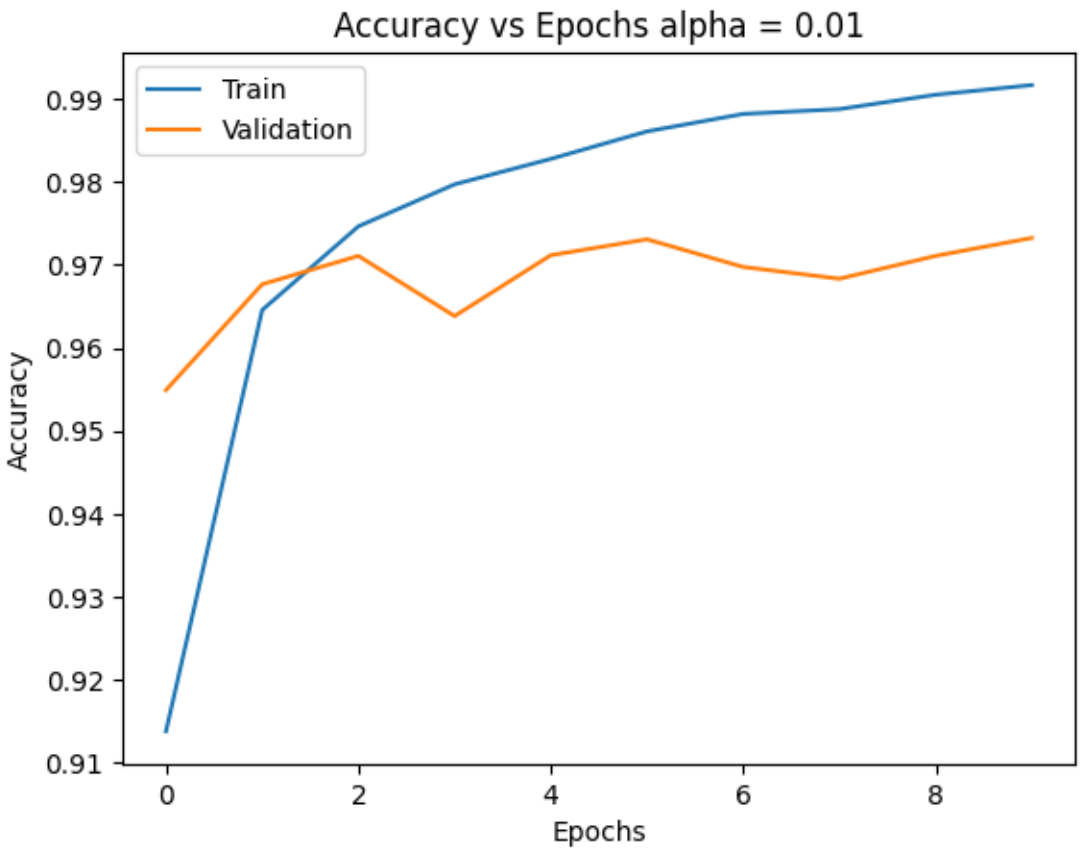
plt.figure()
plt.plot(range(len(accuracy)), accuracy)
plt.plot(range(len(val_accuracy)), val_accuracy)
```

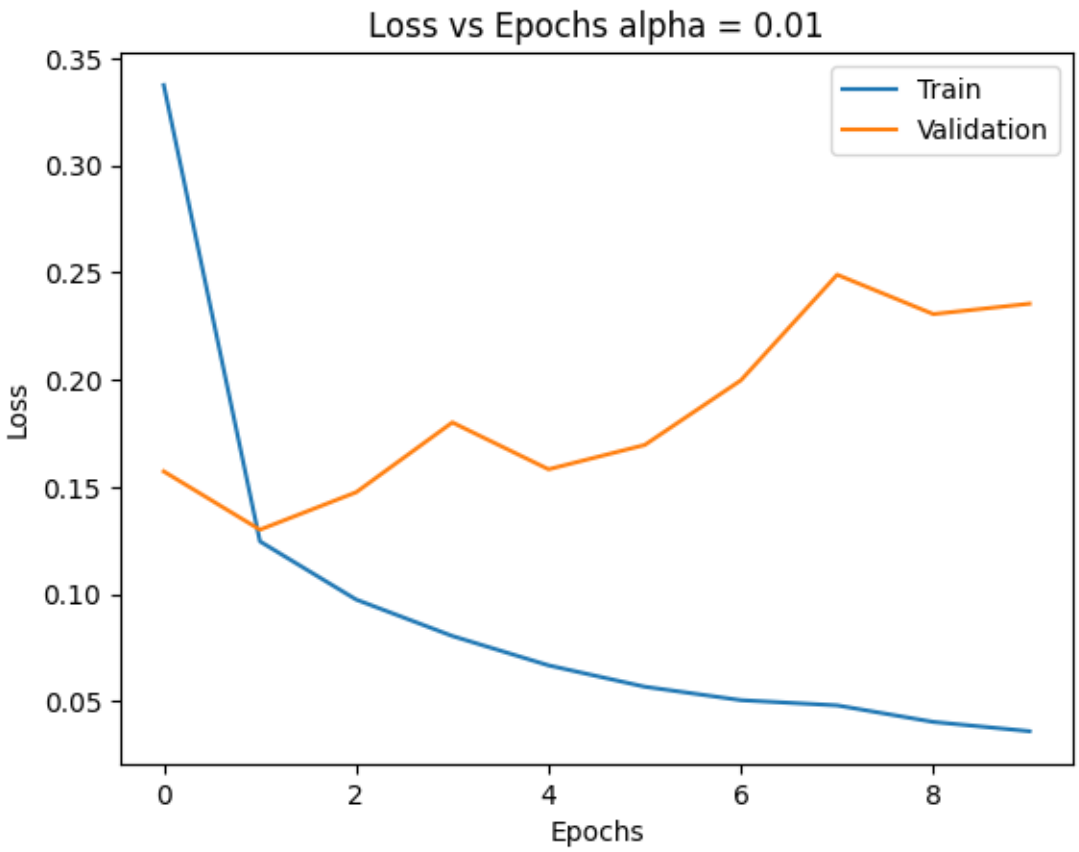
```
plt.title("Accuracy vs Epochs alpha = 0.01")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

loss = history.history['loss']
val_loss = history.history['val_loss']

plt.figure()
plt.plot(range(len(loss)), loss)
plt.plot(range(len(val_loss)), val_loss)
plt.title("Loss vs Epochs alpha = 0.01")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

```
Epoch 1/10
375/375             2s 4ms/step -
accuracy: 0.9138 - loss: 0.3374 - val_accuracy: 0.9549 - val_loss: 0.1573
Epoch 2/10
375/375             1s 3ms/step -
accuracy: 0.9646 - loss: 0.1246 - val_accuracy: 0.9677 - val_loss: 0.1301
Epoch 3/10
375/375             1s 4ms/step -
accuracy: 0.9746 - loss: 0.0975 - val_accuracy: 0.9711 - val_loss: 0.1476
Epoch 4/10
375/375             1s 3ms/step -
accuracy: 0.9797 - loss: 0.0805 - val_accuracy: 0.9638 - val_loss: 0.1801
Epoch 5/10
375/375             1s 3ms/step -
accuracy: 0.9828 - loss: 0.0668 - val_accuracy: 0.9712 - val_loss: 0.1583
Epoch 6/10
375/375             1s 3ms/step -
accuracy: 0.9861 - loss: 0.0569 - val_accuracy: 0.9731 - val_loss: 0.1696
Epoch 7/10
375/375             1s 3ms/step -
accuracy: 0.9882 - loss: 0.0506 - val_accuracy: 0.9697 - val_loss: 0.1998
Epoch 8/10
375/375             1s 3ms/step -
accuracy: 0.9888 - loss: 0.0482 - val_accuracy: 0.9683 - val_loss: 0.2490
Epoch 9/10
375/375             1s 3ms/step -
accuracy: 0.9905 - loss: 0.0405 - val_accuracy: 0.9711 - val_loss: 0.2306
Epoch 10/10
375/375             1s 3ms/step -
accuracy: 0.9917 - loss: 0.0361 - val_accuracy: 0.9732 - val_loss: 0.2355
```


Accuracy vs Epochs alpha = 0.01
Train
Validation
Accuracy
Epochs
0.99
0.98
0.97
0.96
0.95
0.94
0.93
0.92
0.91
0
2
4
6
8



[7]:
```
alpha = 0.001
model = keras.Sequential([
  layers.Dense(512, activation='relu'),
  layers.Dense(10, activation='softmax')
])
model.compile(optimizer=keras.optimizers.RMSprop(alpha),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
history=model.fit(train_images, train_labels,
          epochs=10,
          batch_size=128,
          validation_split=0.2)

accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

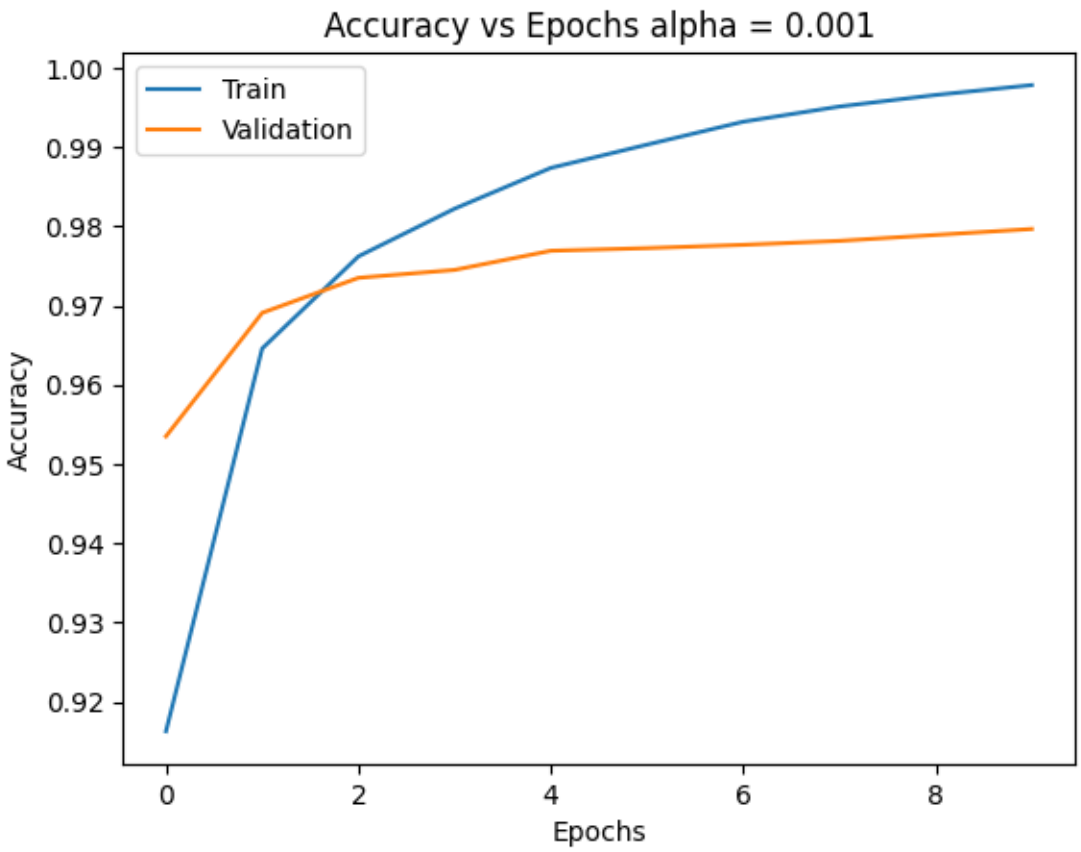
plt.figure()
plt.plot(range(len(accuracy)), accuracy)
plt.plot(range(len(val_accuracy)), val_accuracy)
```

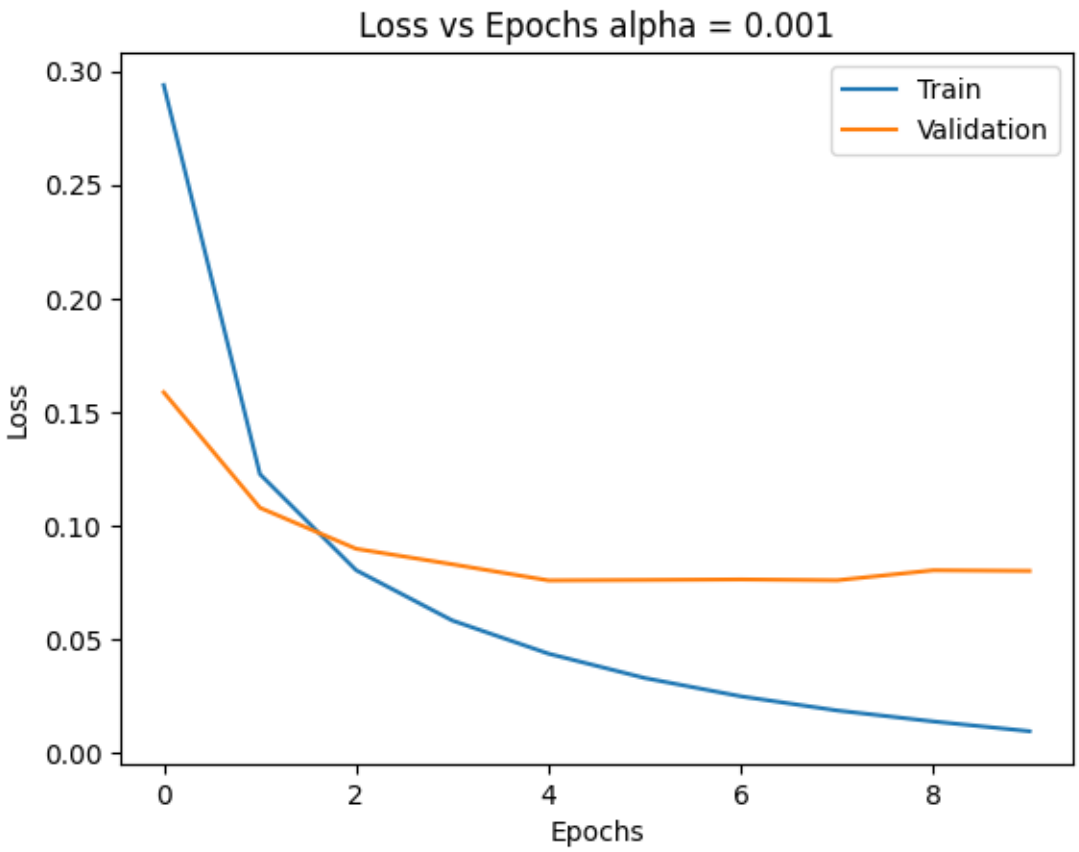
```
plt.title("Accuracy vs Epochs alpha = 0.001")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

loss = history.history['loss']
val_loss = history.history['val_loss']

plt.figure()
plt.plot(range(len(loss)), loss)
plt.plot(range(len(val_loss)), val_loss)
plt.title("Loss vs Epochs alpha = 0.001")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

```
Epoch 1/10
375/375             2s 3ms/step -
accuracy: 0.9163 - loss: 0.2938 - val_accuracy: 0.9535 - val_loss: 0.1587
Epoch 2/10
375/375             1s 3ms/step -
accuracy: 0.9646 - loss: 0.1227 - val_accuracy: 0.9691 - val_loss: 0.1079
Epoch 3/10
375/375             1s 3ms/step -
accuracy: 0.9762 - loss: 0.0804 - val_accuracy: 0.9735 - val_loss: 0.0899
Epoch 4/10
375/375             1s 3ms/step -
accuracy: 0.9822 - loss: 0.0583 - val_accuracy: 0.9745 - val_loss: 0.0831
Epoch 5/10
375/375             1s 3ms/step -
accuracy: 0.9874 - loss: 0.0437 - val_accuracy: 0.9769 - val_loss: 0.0760
Epoch 6/10
375/375             1s 3ms/step -
accuracy: 0.9903 - loss: 0.0330 - val_accuracy: 0.9772 - val_loss: 0.0761
Epoch 7/10
375/375             1s 3ms/step -
accuracy: 0.9932 - loss: 0.0250 - val_accuracy: 0.9777 - val_loss: 0.0764
Epoch 8/10
375/375             1s 3ms/step -
accuracy: 0.9951 - loss: 0.0187 - val_accuracy: 0.9782 - val_loss: 0.0760
Epoch 9/10
375/375             1s 3ms/step -
accuracy: 0.9966 - loss: 0.0139 - val_accuracy: 0.9789 - val_loss: 0.0804
Epoch 10/10
375/375             1s 3ms/step -
accuracy: 0.9978 - loss: 0.0096 - val_accuracy: 0.9797 - val_loss: 0.0801
```


Accuracy vs Epochs alpha = 0.001
Train
Validation
Accuracy
Epochs
1.00
0.99
0.98
0.97
0.96
0.95
0.94
0.93
0.92
0
2
4
6
8



As you decrease the value of alpha (learning rate), the training becomes more stable but slower, the accuracy starts incresing and the loss makes sense.

For alpha = 1.0,

Train accuracy decreases because the weights jump too much and the model cannot learn consistent patterns. Validation accuracy oscillates because predictions change randomly at each update. Train and validation loss go to 0 due to numerical saturation of the softmax, not because the model performs well.

2.0.1 Conclusion

the learning rate is too high, causing unstable updates, numerical issues in the loss, and poor accuracy. The model does not converge and the reported loss values are not reliable.

3 3.2

[8]:
```
model = keras.Sequential([layers.Dense(10, activation='softmax')])
model.compile(optimizer='rmsprop',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```
history_small_model = model.fit(
    train_images, train_labels,
    epochs=50,
    batch_size=128,
    validation_split=0.2)
```

```
Epoch 1/50
375/375             1s 1ms/step -
accuracy: 0.8298 - loss: 0.6833 - val_accuracy: 0.9051 - val_loss: 0.3621
Epoch 2/50
375/375             0s 986us/step -
accuracy: 0.9020 - loss: 0.3534 - val_accuracy: 0.9157 - val_loss: 0.3102
Epoch 3/50
375/375             1s 1ms/step -
accuracy: 0.9121 - loss: 0.3182 - val_accuracy: 0.9198 - val_loss: 0.2929
Epoch 4/50
375/375             0s 968us/step -
accuracy: 0.9154 - loss: 0.3022 - val_accuracy: 0.9220 - val_loss: 0.2856
Epoch 5/50
375/375             0s 950us/step -
accuracy: 0.9186 - loss: 0.2925 - val_accuracy: 0.9233 - val_loss: 0.2783
Epoch 6/50
375/375             0s 933us/step -
accuracy: 0.9202 - loss: 0.2855 - val_accuracy: 0.9245 - val_loss: 0.2749
Epoch 7/50
375/375             0s 1ms/step -
accuracy: 0.9215 - loss: 0.2806 - val_accuracy: 0.9268 - val_loss: 0.2725
Epoch 8/50
375/375             0s 974us/step -
accuracy: 0.9226 - loss: 0.2767 - val_accuracy: 0.9263 - val_loss: 0.2700
Epoch 9/50
375/375             0s 990us/step -
accuracy: 0.9244 - loss: 0.2734 - val_accuracy: 0.9268 - val_loss: 0.2682
Epoch 10/50
375/375             0s 1ms/step -
accuracy: 0.9243 - loss: 0.2709 - val_accuracy: 0.9283 - val_loss: 0.2665
Epoch 11/50
375/375             0s 864us/step -
accuracy: 0.9250 - loss: 0.2685 - val_accuracy: 0.9277 - val_loss: 0.2666
Epoch 12/50
375/375             0s 965us/step -
accuracy: 0.9260 - loss: 0.2664 - val_accuracy: 0.9261 - val_loss: 0.2690
Epoch 13/50
375/375             0s 986us/step -
accuracy: 0.9266 - loss: 0.2647 - val_accuracy: 0.9296 - val_loss: 0.2650
Epoch 14/50
375/375             0s 795us/step -
accuracy: 0.9276 - loss: 0.2631 - val_accuracy: 0.9287 - val_loss: 0.2645
```

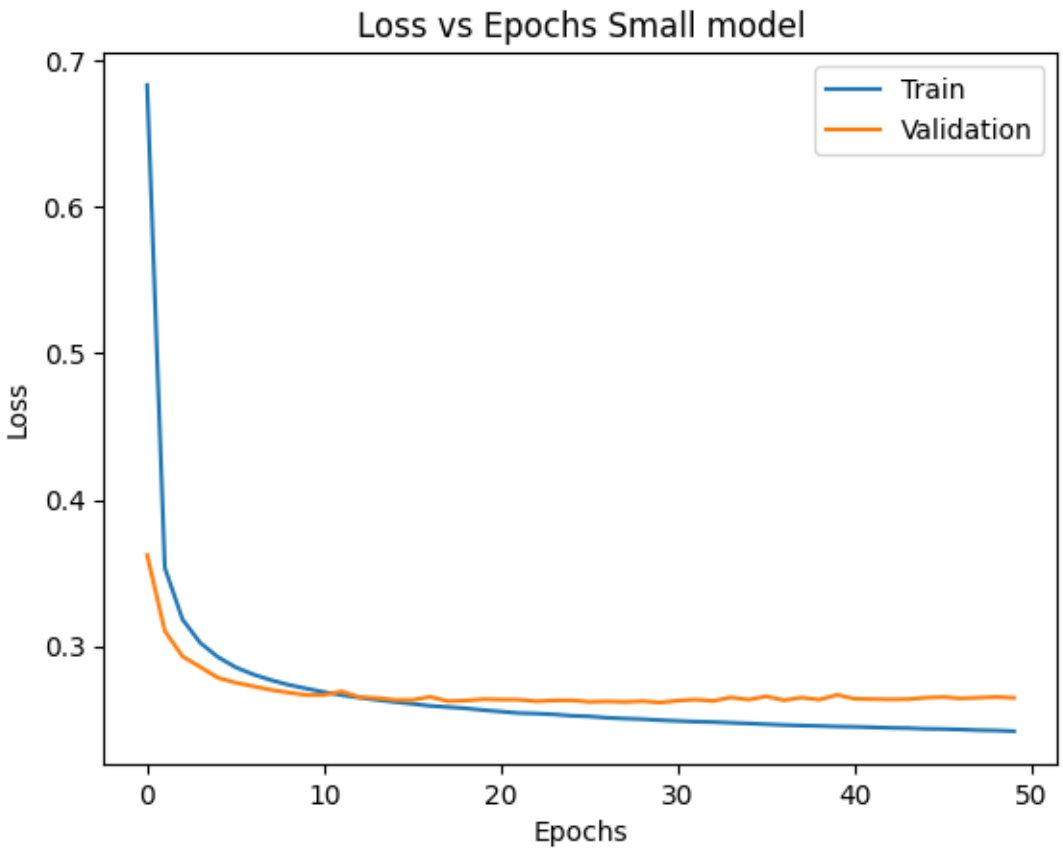
```
Epoch 15/50
375/375             0s 810us/step -
accuracy: 0.9277 - loss: 0.2617 - val_accuracy: 0.9295 - val_loss: 0.2631
Epoch 16/50
375/375             0s 860us/step -
accuracy: 0.9279 - loss: 0.2605 - val_accuracy: 0.9295 - val_loss: 0.2631
Epoch 17/50
375/375             0s 821us/step -
accuracy: 0.9288 - loss: 0.2591 - val_accuracy: 0.9277 - val_loss: 0.2653
Epoch 18/50
375/375             0s 819us/step -
accuracy: 0.9287 - loss: 0.2583 - val_accuracy: 0.9293 - val_loss: 0.2623
Epoch 19/50
375/375             0s 950us/step -
accuracy: 0.9291 - loss: 0.2574 - val_accuracy: 0.9297 - val_loss: 0.2627
Epoch 20/50
375/375             0s 850us/step -
accuracy: 0.9296 - loss: 0.2562 - val_accuracy: 0.9291 - val_loss: 0.2636
Epoch 21/50
375/375             0s 862us/step -
accuracy: 0.9300 - loss: 0.2552 - val_accuracy: 0.9301 - val_loss: 0.2633
Epoch 22/50
375/375             0s 862us/step -
accuracy: 0.9299 - loss: 0.2542 - val_accuracy: 0.9302 - val_loss: 0.2633
Epoch 23/50
375/375             0s 944us/step -
accuracy: 0.9304 - loss: 0.2538 - val_accuracy: 0.9308 - val_loss: 0.2622
Epoch 24/50
375/375             0s 904us/step -
accuracy: 0.9302 - loss: 0.2533 - val_accuracy: 0.9307 - val_loss: 0.2628
Epoch 25/50
375/375             0s 872us/step -
accuracy: 0.9309 - loss: 0.2523 - val_accuracy: 0.9302 - val_loss: 0.2629
Epoch 26/50
375/375             0s 856us/step -
accuracy: 0.9308 - loss: 0.2519 - val_accuracy: 0.9301 - val_loss: 0.2618
Epoch 27/50
375/375             0s 921us/step -
accuracy: 0.9319 - loss: 0.2509 - val_accuracy: 0.9303 - val_loss: 0.2622
Epoch 28/50
375/375             0s 849us/step -
accuracy: 0.9318 - loss: 0.2504 - val_accuracy: 0.9305 - val_loss: 0.2618
Epoch 29/50
375/375             0s 812us/step -
accuracy: 0.9314 - loss: 0.2500 - val_accuracy: 0.9297 - val_loss: 0.2623
Epoch 30/50
375/375             0s 812us/step -
accuracy: 0.9319 - loss: 0.2493 - val_accuracy: 0.9307 - val_loss: 0.2613
```

```
Epoch 31/50
375/375             0s 964us/step -
accuracy: 0.9326 - loss: 0.2488 - val_accuracy: 0.9308 - val_loss: 0.2627
Epoch 32/50
375/375             0s 861us/step -
accuracy: 0.9317 - loss: 0.2483 - val_accuracy: 0.9308 - val_loss: 0.2633
Epoch 33/50
375/375             0s 828us/step -
accuracy: 0.9321 - loss: 0.2480 - val_accuracy: 0.9309 - val_loss: 0.2625
Epoch 34/50
375/375             0s 776us/step -
accuracy: 0.9325 - loss: 0.2475 - val_accuracy: 0.9297 - val_loss: 0.2649
Epoch 35/50
375/375             0s 763us/step -
accuracy: 0.9332 - loss: 0.2471 - val_accuracy: 0.9302 - val_loss: 0.2633
Epoch 36/50
375/375             0s 847us/step -
accuracy: 0.9326 - loss: 0.2465 - val_accuracy: 0.9302 - val_loss: 0.2656
Epoch 37/50
375/375             0s 766us/step -
accuracy: 0.9325 - loss: 0.2460 - val_accuracy: 0.9310 - val_loss: 0.2629
Epoch 38/50
375/375             0s 762us/step -
accuracy: 0.9327 - loss: 0.2456 - val_accuracy: 0.9308 - val_loss: 0.2648
Epoch 39/50
375/375             0s 973us/step -
accuracy: 0.9333 - loss: 0.2454 - val_accuracy: 0.9315 - val_loss: 0.2633
Epoch 40/50
375/375             0s 904us/step -
accuracy: 0.9335 - loss: 0.2450 - val_accuracy: 0.9293 - val_loss: 0.2666
Epoch 41/50
375/375             0s 919us/step -
accuracy: 0.9339 - loss: 0.2448 - val_accuracy: 0.9307 - val_loss: 0.2639
Epoch 42/50
375/375             0s 1ms/step -
accuracy: 0.9328 - loss: 0.2444 - val_accuracy: 0.9308 - val_loss: 0.2637
Epoch 43/50
375/375             0s 1ms/step -
accuracy: 0.9342 - loss: 0.2441 - val_accuracy: 0.9308 - val_loss: 0.2635
Epoch 44/50
375/375             0s 1ms/step -
accuracy: 0.9335 - loss: 0.2439 - val_accuracy: 0.9290 - val_loss: 0.2637
Epoch 45/50
375/375             0s 898us/step -
accuracy: 0.9341 - loss: 0.2434 - val_accuracy: 0.9308 - val_loss: 0.2647
Epoch 46/50
375/375             0s 927us/step -
accuracy: 0.9336 - loss: 0.2433 - val_accuracy: 0.9304 - val_loss: 0.2651
```

```
Epoch 47/50
375/375             0s 864us/step -
accuracy: 0.9332 - loss: 0.2429 - val_accuracy: 0.9308 - val_loss: 0.2642
Epoch 48/50
375/375             0s 835us/step -
accuracy: 0.9342 - loss: 0.2425 - val_accuracy: 0.9303 - val_loss: 0.2647
Epoch 49/50
375/375             0s 838us/step -
accuracy: 0.9340 - loss: 0.2423 - val_accuracy: 0.9286 - val_loss: 0.2652
Epoch 50/50
375/375             0s 861us/step -
accuracy: 0.9343 - loss: 0.2418 - val_accuracy: 0.9308 - val_loss: 0.2646
```

[10]:
```
loss = history_small_model.history['loss']
val_loss = history_small_model.history['val_loss']

plt.figure()
plt.plot(range(len(loss)), loss)
plt.plot(range(len(val_loss)), val_loss)
plt.title("Loss vs Epochs Small model")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```



The validation loss matches the training loss until around epoch 15 where the train loss decrease slightly more than the validation loss, but there is no clear sign of overfitting.

4 3.3

[11]:
```
model = keras.Sequential([
  layers.Dense(96, activation='relu'),
  layers.Dense(96, activation='relu'),
  layers.Dense(10, activation='softmax'),
])
model.compile(optimizer='rmsprop',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
history_large_model = model.fit(
    train_images, train_labels,
    epochs=50,
    batch_size=128,
    validation_split=0.2)
```

```
Epoch 1/50
```

```
375/375            1s 2ms/step -
accuracy: 0.8982 - loss: 0.3610 - val_accuracy: 0.9463 - val_loss: 0.1834
Epoch 2/50
375/375            1s 2ms/step -
accuracy: 0.9535 - loss: 0.1586 - val_accuracy: 0.9547 - val_loss: 0.1537
Epoch 3/50
375/375            1s 2ms/step -
accuracy: 0.9671 - loss: 0.1111 - val_accuracy: 0.9672 - val_loss: 0.1107
Epoch 4/50
375/375            1s 2ms/step -
accuracy: 0.9739 - loss: 0.0858 - val_accuracy: 0.9698 - val_loss: 0.1029
Epoch 5/50
375/375            1s 2ms/step -
accuracy: 0.9792 - loss: 0.0690 - val_accuracy: 0.9705 - val_loss: 0.0989
Epoch 6/50
375/375            1s 2ms/step -
accuracy: 0.9830 - loss: 0.0569 - val_accuracy: 0.9722 - val_loss: 0.0913
Epoch 7/50
375/375            1s 2ms/step -
accuracy: 0.9855 - loss: 0.0481 - val_accuracy: 0.9739 - val_loss: 0.0900
Epoch 8/50
375/375            1s 2ms/step -
accuracy: 0.9880 - loss: 0.0396 - val_accuracy: 0.9738 - val_loss: 0.0888
Epoch 9/50
375/375            1s 2ms/step -
accuracy: 0.9901 - loss: 0.0325 - val_accuracy: 0.9718 - val_loss: 0.0932
Epoch 10/50
375/375            1s 2ms/step -
accuracy: 0.9920 - loss: 0.0282 - val_accuracy: 0.9726 - val_loss: 0.1007
Epoch 11/50
375/375            1s 2ms/step -
accuracy: 0.9921 - loss: 0.0240 - val_accuracy: 0.9693 - val_loss: 0.1157
Epoch 12/50
375/375            1s 2ms/step -
accuracy: 0.9939 - loss: 0.0200 - val_accuracy: 0.9682 - val_loss: 0.1245
Epoch 13/50
375/375            1s 2ms/step -
accuracy: 0.9952 - loss: 0.0157 - val_accuracy: 0.9740 - val_loss: 0.1131
Epoch 14/50
375/375            1s 2ms/step -
accuracy: 0.9955 - loss: 0.0148 - val_accuracy: 0.9737 - val_loss: 0.1074
Epoch 15/50
375/375            1s 2ms/step -
accuracy: 0.9958 - loss: 0.0134 - val_accuracy: 0.9756 - val_loss: 0.1148
Epoch 16/50
375/375            1s 2ms/step -
accuracy: 0.9969 - loss: 0.0102 - val_accuracy: 0.9770 - val_loss: 0.1167
Epoch 17/50
```

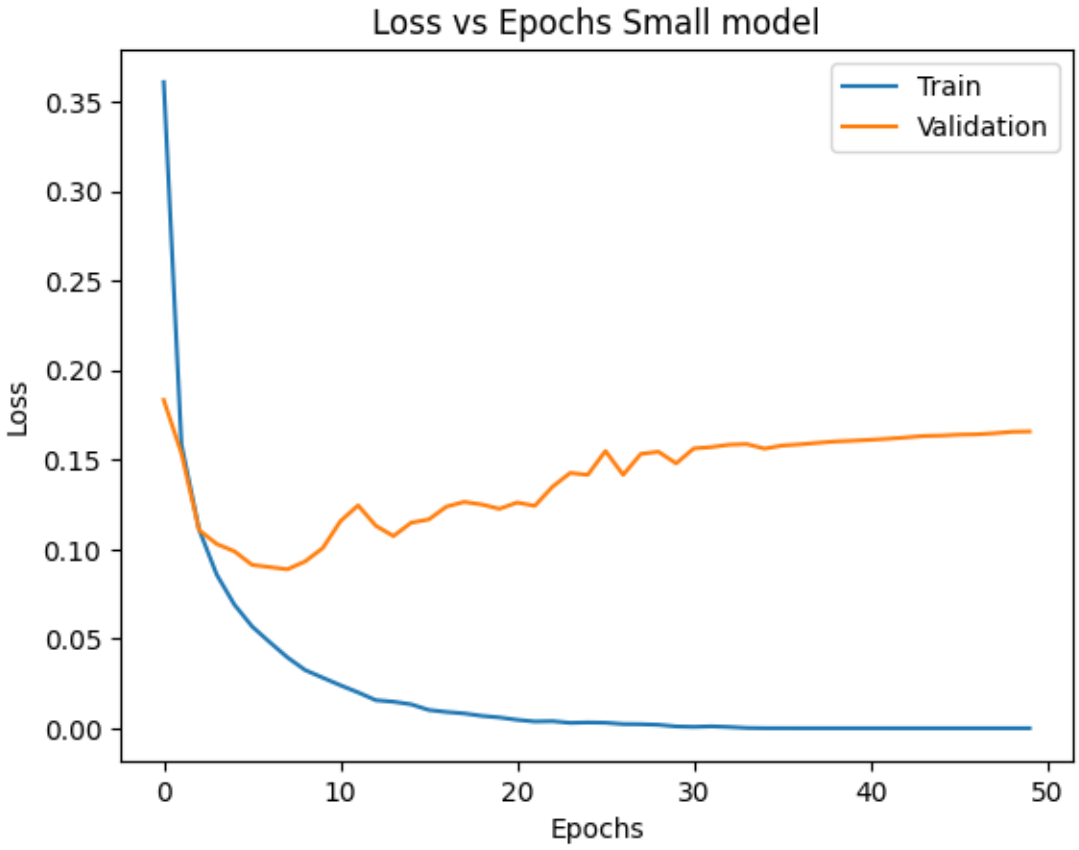
```
375/375            1s 2ms/step -
accuracy: 0.9969 - loss: 0.0091 - val_accuracy: 0.9746 - val_loss: 0.1238
Epoch 18/50
375/375            1s 2ms/step -
accuracy: 0.9975 - loss: 0.0083 - val_accuracy: 0.9748 - val_loss: 0.1265
Epoch 19/50
375/375            1s 2ms/step -
accuracy: 0.9981 - loss: 0.0069 - val_accuracy: 0.9760 - val_loss: 0.1250
Epoch 20/50
375/375            1s 2ms/step -
accuracy: 0.9982 - loss: 0.0060 - val_accuracy: 0.9751 - val_loss: 0.1226
Epoch 21/50
375/375            1s 2ms/step -
accuracy: 0.9986 - loss: 0.0047 - val_accuracy: 0.9765 - val_loss: 0.1261
Epoch 22/50
375/375            1s 2ms/step -
accuracy: 0.9990 - loss: 0.0038 - val_accuracy: 0.9766 - val_loss: 0.1243
Epoch 23/50
375/375            1s 2ms/step -
accuracy: 0.9989 - loss: 0.0039 - val_accuracy: 0.9762 - val_loss: 0.1349
Epoch 24/50
375/375            1s 2ms/step -
accuracy: 0.9991 - loss: 0.0030 - val_accuracy: 0.9753 - val_loss: 0.1426
Epoch 25/50
375/375            1s 2ms/step -
accuracy: 0.9990 - loss: 0.0032 - val_accuracy: 0.9769 - val_loss: 0.1416
Epoch 26/50
375/375            1s 2ms/step -
accuracy: 0.9991 - loss: 0.0031 - val_accuracy: 0.9761 - val_loss: 0.1549
Epoch 27/50
375/375            1s 2ms/step -
accuracy: 0.9992 - loss: 0.0023 - val_accuracy: 0.9761 - val_loss: 0.1416
Epoch 28/50
375/375            1s 2ms/step -
accuracy: 0.9993 - loss: 0.0023 - val_accuracy: 0.9758 - val_loss: 0.1533
Epoch 29/50
375/375            1s 2ms/step -
accuracy: 0.9995 - loss: 0.0019 - val_accuracy: 0.9765 - val_loss: 0.1545
Epoch 30/50
375/375            1s 2ms/step -
accuracy: 0.9998 - loss: 0.0010 - val_accuracy: 0.9770 - val_loss: 0.1480
Epoch 31/50
375/375            1s 2ms/step -
accuracy: 0.9999 - loss: 7.7410e-04 - val_accuracy: 0.9772 - val_loss: 0.1563
Epoch 32/50
375/375            1s 2ms/step -
accuracy: 0.9997 - loss: 0.0011 - val_accuracy: 0.9762 - val_loss: 0.1571
Epoch 33/50
```

```
375/375            1s 2ms/step -
accuracy: 0.9998 - loss: 6.9911e-04 - val_accuracy: 0.9773 - val_loss: 0.1584
Epoch 34/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 1.7808e-04 - val_accuracy: 0.9781 - val_loss: 0.1587
Epoch 35/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 4.2549e-05 - val_accuracy: 0.9776 - val_loss: 0.1562
Epoch 36/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 3.0053e-05 - val_accuracy: 0.9780 - val_loss: 0.1579
Epoch 37/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 2.5126e-05 - val_accuracy: 0.9780 - val_loss: 0.1586
Epoch 38/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 2.2235e-05 - val_accuracy: 0.9778 - val_loss: 0.1594
Epoch 39/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 2.0668e-05 - val_accuracy: 0.9777 - val_loss: 0.1602
Epoch 40/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 1.9034e-05 - val_accuracy: 0.9781 - val_loss: 0.1606
Epoch 41/50
375/375            1s 1ms/step -
accuracy: 1.0000 - loss: 1.7767e-05 - val_accuracy: 0.9780 - val_loss: 0.1612
Epoch 42/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 1.6520e-05 - val_accuracy: 0.9780 - val_loss: 0.1617
Epoch 43/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 1.5941e-05 - val_accuracy: 0.9782 - val_loss: 0.1625
Epoch 44/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 1.4983e-05 - val_accuracy: 0.9778 - val_loss: 0.1633
Epoch 45/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 1.4348e-05 - val_accuracy: 0.9777 - val_loss: 0.1635
Epoch 46/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 1.3760e-05 - val_accuracy: 0.9778 - val_loss: 0.1640
Epoch 47/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 1.3123e-05 - val_accuracy: 0.9778 - val_loss: 0.1642
Epoch 48/50
375/375            1s 2ms/step -
accuracy: 1.0000 - loss: 1.2658e-05 - val_accuracy: 0.9777 - val_loss: 0.1648
Epoch 49/50
```

```
375/375              1s 2ms/step -
accuracy: 1.0000 - loss: 1.2206e-05 - val_accuracy: 0.9779 - val_loss: 0.1656
Epoch 50/50
375/375              1s 2ms/step -
accuracy: 1.0000 - loss: 1.1763e-05 - val_accuracy: 0.9778 - val_loss: 0.1658
```

[12]:
```
loss = history_large_model.history['loss']
val_loss = history_large_model.history['val_loss']

plt.figure()
plt.plot(range(len(loss)), loss)
plt.plot(range(len(val_loss)), val_loss)
plt.title("Loss vs Epochs Small model")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```



The validation loss hits its minimum at around epoch 10 where the model start overfitting the training data.

From epoch 10, the training loss continue to decrease while the validation loss increases which is a clear sign of model overfitting.

[ ]:

[ ]:

[ ]: